Unidad 3 / Escenario 6

Lectura fundamental

Contenedores estándar de datos no lineales

Contenido

1 Conjuntos 1

2 Mapas 3

3 Problemas propuestos 8

Palabras claves: ArrayList, LinkedList, Stack y Queue.

En escenarios anteriores usted se familiarizó y puso en práctica el uso de arreglos de tamaño fijo y de algunos contenedores estándar de datos de los lenguajes de programación, tales como listas, pilas y colas. En este Escenario, usted conocerá y utilizará otros contenedores estándar de datos de los lenguajes de programación de alto nivel y de propósito general: los contenedores tipo conjunto y los tipo función. Ambos son abstracciones de lo que matemáticamente es un conjunto y una función, respectivamente.

Analice paso a paso los ejemplos de código presentados con diferentes entradas de datos y a que use datos repetidos en el análisis del código, use contenedores fuertemente relacionados con los conceptos de conjuntos y funciones en matemáticas. El propósito es que también pruebe los contenedores con tipos de datos *double* (a través de su clase envolvente: *Double*) y con cadenas (*String*), para una mayor apropiación del conocimiento.

1. Conjuntos

Los conjuntos, definidos formalmente, son una colección de elementos que no tienen un orden particular y que no contienen elementos repetidos. Al ser un tipo de comportamiento usado regularmente en el desarrollo de programas, los lenguajes de programación de propósito general y de alto nivel tienen al menos dos implementaciones estándar de estructuras de datos que abstraen el comportamiento de lo que es un conjunto. En Java, estas herramientas son los *TreeSet* y los *HashSet*. La particularidad de los conjuntos consiste en que se pueden agregar tantos elementos como se desee, pero no se almacenan dos elementos que se consideren iguales; esto, para respetar la definición de conjunto. Tanto los *TreeSet* como los *HashSet* son colecciones estándar de Java parametrizadas, por lo que se pueden almacenar elementos de tipos arbitrarios. En este documento se exponen ejemplos usando enteros, pero su empleo se extiende a cualquier tipo de dato nativo, estándar o definido por el programador, siempre y cuando se cumplan algunos requisitos específicos para cuando se use un *TreeSet* y un *HashSet*.

1.1. *TreeSet*

Los *TreeSet* en Java son una implementación del tipo abstracto de dato conjunto mediante un árbol. Aunque los programadores de aplicaciones que usan Java no tienen control sobre la representación interna del conjunto, el conocimiento de cómo está hecho es importante puesto que da una clara idea de las complejidades computacionales de las operaciones que se pueden hacer con los contenedores (Stack Overflow Community, 2018). Es necesario investigar a profundidad estos temas para adquirir una mayor eficiencia en sus implementaciones.

Una ventaja de los *TreeSet* es que, al estar implementados internamente como un árbol, su impresión y recorrido se hace de manera ascendente respecto al orden natural de los elementos que almacena. En el Código 1 se puede ver que, en las impresiones de datos, los números siempre salen en orden ascendente. Una restricción en el uso de los *TreeSet* tiene que ver con la necesidad de que los datos que almacena deben tener un orden establecido, el cual está por defecto en las clases envolventes y clases estándar de Java, como *Double*, *String, Integer, Long*, etc.

```java
import java.util.Scanner;
import java.util.TreeSet;
public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        TreeSet<Integer> set = new TreeSet<Integer>();
        System.out.print("How many initial integers? ");
        int n = sn.nextInt();
        for( int i = 0; i < n; ++ i ){
            System.out.print("Type an integer: ");
            int tmp = sn.nextInt();
            set.add(tmp);
        }

        System.out.println("Set printed as String: "+set);

        System.out.println("Set traversed by a for-each loop: ");
        for( int k: set )
            System.out.print(k+" ");
        System.out.println();

        System.out.print("Type an integer to search: ");
        int q = sn.nextInt();
        boolean isIn = set.contains(q);
        System.out.println(q +(isIn?" is":" is not")+" on the set: "+set);

        System.out.print("Type an element to remove: ");
        int r = sn.nextInt();
        set.remove(r);
        System.out.println("Set after removing "+r+": "+set  );
    }
}
```

Código 1: Ejemplo de uso de TreeSet

1.2. HashSet

Los comportamientos del *HashSet* son casi idénticos a los comportamientos del *TreeSet*. En particular, ambos abstraen lo que es un conjunto formalmente en matemáticas: una colección de elementos sin repeticiones. El *HashSet* está implementado internamente con una tabla *hash*. El funcionamiento interno del *HashSet* se escapa del objetivo de este Módulo, pero, al igual que con el *TreeSet*, se insta al estudiante a indagar más al respecto para un mayor entendimiento de sus comportamientos y complejidades.

El Código 2 muestra un programa casi idéntico al que se puede ver en el Código 1; sin embargo, es probable que para las mismas entradas produzca salidas diferentes en las impresiones, a la salida estándar del set completo. Esto debido a su implementación interna: el *hashset* no almacena los datos en un orden específico, por lo que al imprimirlo o recorrerlo el resultado y orden en que se ven los datos no es necesariamente ascendente, como sí lo es en el caso de los *TreeSet*.

```java
import java.util.Scanner;
import java.util.HashSet;
public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        HashSet<Integer> set = new HashSet<Integer>();
        System.out.print("How many initial integers? ");
        int n = sn.nextInt();
        for( int i = 0; i < n; ++ i ){
            System.out.print("Type an integer: ");
            int tmp = sn.nextInt();
            set.add(tmp);
        }

        System.out.println("Set printed as String: "+set);

        System.out.println("Set traversed by a for-each loop: ");
        for( int k: set )
            System.out.print(k+" ");
        System.out.println();

        System.out.print("Type an integer to search: ");
        int q = sn.nextInt();
        boolean isIn = set.contains(q);
        System.out.println(q +(isIn?" is":" is not")+" on the set: "+set);

        System.out.print("Type an element to remove: ");
        int r = sn.nextInt();
        set.remove(r);
        System.out.println("Set after removing "+r+": "+set  );
    }
}
```

Código 2: Ejemplo de uso de *HashSet*

2. Mapas

Las funciones en matemáticas (conocidas también como mapas o mapeos entre dos conjuntos) son una relación entre dos conjuntos: el dominio y el codominio (que pueden ser iguales o no), en donde cada elemento del dominio está relacionado con uno y solo un elemento del codominio. En los lenguajes de programación, una manera de abstraerlos es a través de los mapas, también conocidos como diccionarios en algunos lenguajes de programación como C#.

Dada la definición de función, se puede ver que cuando un elemento x del dominio de una función está relacionado con algún elemento del codominio, ese elemento *x* no se puede relacionar con otro elemento del codominio o se violaría la definición de función. En el caso del codominio, varios elementos de este pueden estar relacionados con el mismo elemento en el dominio. Por estas razones, a los elementos del dominio en un mapa se les suele llamar llaves.

En Java se tienen, de manera estándar, implementados los mapas mediante tablas *hash* y árboles para almacenamiento de las llaves. Es importante tener en cuenta que el método de almacenamiento para garantizar unicidad en el dominio es el mismo para el *TreeSet* y para *TreeMap* y a su vez es el mismo para el *HashSet* y para el *HashMap*.

2.1. TreeMap

Los *TreeMap* en Java son la implementación de diccionarios vía árboles ordenados para el conjunto de llaves (o dominio). Aunque su declaración e instanciación se pueden hacer de varias formas, una de las más comunes es especificar en ambos casos tanto el dominio como el codominio. Ejemplo, un diccionario con dominio en los números enteros y codominio en las cadenas de caracteres se declararía e iniciaría como se puede ver en el código 3, en la segunda línea, en el método *main*.

En el ejemplo de uso de *TreeMap* del código 3 se crea un diccionario en donde las llaves son números enteros, que bien podrían ser números de identificación de personas. Para simplificar, el valor asociado a cada llave será de tipo *String* y representará un nombre de una persona. Al ejecutar el programa se puede ver que la pareja (421, Ralph), es decir, la llave 421 asociada al valor Ralph en el diccionario, es reemplazada por la pareja (421, Elisabeth). Este reemplazo se puede evidenciar al momento de imprimir el diccionario usando el *foreach*. Este reemplazo se da puesto que un *Map* no almacena dos parejas con la misma llave; es por esto que la segunda pareja introducida reemplaza la existente.

El recorrido del mapa se hace a través de la estructura repetitiva *for-each*, transitando el conjunto de llaves. Se puede ver en la ejecución del código que, sin importar el orden en que se introducen las parejas al mapa, su impresión se hace de manera ascendente respecto al orden natural de las llaves. También se puede, para propósitos de depuración, sencillamente imprimir todo el mapa, como se aprecia justo antes del ciclo *for* en el código. Además, en el programa del Código 3 se solicita al usuario un *id* a buscar y, de encontrarlo en el mapa, muestra el valor asociado a él. Finalmente, el programa pide un valor a eliminar y, al encontrarlo, lo hace. Es importante compilar y ejecutar el programa con diferentes entradas posibles para apropiarse del funcionamiento y uso de las principales funcionalidades de los *TreeMap*. De igual manera, puede revisar la documentación de Java para más funcionalidades (Oracle, 1999).

```java
import java.util.Scanner;
import java.util.TreeMap;
public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        TreeMap<Integer,String> idToName = new TreeMap<Integer,String>();
        idToName.put(923,"John");
        idToName.put(421,"Ralph");
        idToName.put(238,"Albert");
        idToName.put(529,"Mary");
        idToName.put(421,"Elisabeth");
        idToName.put(101,"Caroline");

        System.out.println(idToName);

        for( int key : idToName.keySet() )
            System.out.println(key+" maps to: "+idToName.get(key));

        System.out.print("Type an integer id to search: ");
        int idToSearch = sn.nextInt();
        boolean isIdIn = idToName.containsKey(idToSearch);
        if( isIdIn )
            System.out.println("The person with id="+
                    idToSearch+" is "+idToName.get(idToSearch));
        else
            System.out.println("There is no person with id="+idToSearch);

        System.out.print("Type an integer id to delete: ");
        int idToDelete = sn.nextInt();
        boolean isIn = idToName.containsKey(idToDelete);
        if( isIn )
        {
            System.out.println(idToName.get(idToDelete)+" removed.");
            idToName.remove(idToDelete);
        }
        else
            System.out.println("There is no person with id="+idToDelete);

        System.out.println(idToName);
    }
}
```

Código 3: Ejemplo de uso de *TreeMap*

2.2. *HashMap*

Finalmente, el último contenedor que usará en esta sección, es el de tipo *HashMap*, el cual tiene muchos de los métodos y funcionalidades de los *TreeMap*. Su diferencia principal radica en la forma en que se implementan internamente. Esta implementación hace que, a diferencia del *TreeMap*, sus llaves no se almacenen necesariamente en algún orden específico respecto a ellas. Vale la pena aclarar que los *HashMap* bien usados tienen un mejor rendimiento que los *TreeMap*, aunque una de las principales ventajas de estos últimos es el hecho de que su recorrido se hace en orden con respecto al conjunto de llaves de las parejas almacenadas.

En el programa del Código 4, que es básicamente el mismo presentado en el Código 3, pero usando un *HashMap* en lugar de un *TreeMap*, se puede observar que al ejecutarse la impresión de las llaves no tiene ningún orden en particular en relación con las llaves de las parejas usadas. Otra diferencia entre ellos es que al instanciar el *HashMap* se pasa como argumento un número entero, en este caso 10. Este número sirve para la construcción de la estructura interna del mapa. En general, se desea que este número sea igual o superior a la cantidad máxima de elementos que se espera almacenar en el *HashMap* para evitar un fenómeno conocido como colisiones en mapas hechos a bases de tablas *hash*, como es el caso de los *HashMap* de Java. Para profundizar este tema, revise aspectos y detalles de implementación de tablas hash en Cormen, Leiserson, Rivest y Stein (2009).

```java
import java.util.Scanner;
import java.util.HashMap;
public class Main {
    public static void main( String args[] ){
        Scanner sn = new Scanner( System.in );
        HashMap<Integer,String> idToName = new HashMap<Integer,String>(10);
        idToName.put(923,"John");
        idToName.put(421,"Ralph");
        idToName.put(238,"Albert");
        idToName.put(529,"Mary");
        idToName.put(421,"Elisabeth");
        idToName.put(101,"Caroline");

        System.out.println(idToName);
        for( int key : idToName.keySet() )
            System.out.println(key+" maps to: "+idToName.get(key));

        System.out.print("Type an integer id to search: ");
        int idToSearch = sn.nextInt();
        boolean isIdIn = idToName.containsKey(idToSearch);

        if( isIdIn )
            System.out.println("The person with id="+
                    idToSearch+" is "+idToName.get(idToSearch));
        else
            System.out.println("There is no person with id="+idToSearch);

        System.out.print("Type an integer id to delete: ");
        int idToDelete = sn.nextInt();
        boolean isIn = idToName.containsKey(idToDelete);

        if( isIn )
        {
            System.out.println(idToName.get(idToDelete)+" removed.");
            idToName.remove(idToDelete);
        }
        else
            System.out.println("There is no person with id="+idToDelete);

        System.out.println(idToName);
    }
}
```

Código 4: Ejemplo de uso de *HashMap*

3. Problemas propuestos

Casi todos problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java que tome como entrada ciertos datos leídos desde su entrada estándar y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con los datos de entrada. En todos los problemas propuestos se debe asumir que las entradas cumplirán con lo estipulado en la sección "`Descripción de entrada`" de cada problema. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo estipulado en la sección "`Descripción de salida`" y que sea únicamente lo solicitado allí, nada más.

Teniendo en cuenta los contenedores no lineales tipo conjunto o función (*Sets* y *Maps*), se espera que para solucionarlos se usen estos de alguna manera. Como requisito adicional, toda la lectura de datos (uso de *Scanner*) e impresión de datos a la salida estándar (uso de *System.out*) debe hacerse en el método *main*; es decir, en los demás métodos no puede hacer ni lectura ni impresión de datos.

Los siguientes ejercicios se pueden resolver usando arreglos de tamaño fijo o contenedores lineales y empleando los contenedores.

1. **Contador de dígitos**: diseñe e implemente un programa en Java que responda la siguiente pregunta: si quisiera escribir con lápiz y papel los números enteros desde el 0 hasta el ***n*** (***n*** es un argumento que lee el programa, ¿cuántas veces escribiría cada dígito del 0 al 9?

- Descripción de entrada: un número entero entre 0 y 10000.
- Descripción de salida: 10 líneas, una para cada dígito del 0 al 9, indicando el número de veces que se escribiría el dígito al escribir los números del 0 al ***n***.
- Ejemplo de entrada:
  12
- Ejemplo de salida:
  El dígito 0 se escribiría 2 veces.
  El dígito 1 se escribiría 5 veces.
  El dígito 2 se escribiría 2 veces.
  El dígito 3 se escribiría 1 veces.
  El dígito 4 se escribiría 1 veces.
  El dígito 5 se escribiría 1 veces.
  El dígito 6 se escribiría 1 veces.
  El dígito 7 se escribiría 1 veces.
  El dígito 8 se escribiría 1 veces.
  El dígito 9 se escribiría 1 veces.

2. **Centro de masas**: dados un conjunto de puntos con masa en el plano, hacer un programa que lea estos datos y calcule y muestre el centro de masas de estos puntos. El centro de masas está dado por la siguiente fórmula:

$$x_{cm} = \frac{\sum_{i=1}^{n} m_i * x_i}{\sum_{i=1}^{n} m_i}$$

$$y_{cm} = \frac{\sum_{i=1}^{n} m_i * y_i}{\sum_{i=1}^{n} m_i}$$

- Descripción de entrada: un número entero ***n*** seguido de ***n*** triplas de números. Cada tripla es un punto con su masa, primero la coordenada ***x***, luego la coordenada ***y*** y luego la masa ***m*** para ese punto.

- Descripción de salida: dos números, la coordenada ***x*** del centro de masa y la coordenada y del centro de masa de los puntos leídos.

- Ejemplo de entrada:
  5
  10 10 2
  10 20 4
  10 30 8
  10 40 1
  10 50 16

- Ejemplo de salida:
  10 38.065

3. **Espectro**: diseñar e implementar un programa que tome como entrada 5 números enteros positivos y produzca como salida un patrón, como se muestra en el ejemplo.

- Descripción de entrada: un número **n** entero positivo seguido de **n** números enteros no negativos. Todos entre 0 y 100.
- Descripción de salida: el patrón según se muestra en el ejemplo. El primer número leído **n** será la cantidad de columnas de la matriz a imprimir. Los siguientes **n** números serán la cantidad de numerales que salen en cada columna. Así, en el ejemplo, en la primera columna hay 3 numerales, en la segunda un numeral, en la tercera 8 (el mayor valor da la cantidad de filas), la penúltima columna muestra 4 numerales y la última columna muestra 2 numerales.

- Ejemplo de entrada:
  5
  3 1 8 4 2
- Ejemplo de salida:

```
..#..
..#..
..#..
..#..
..#..
..##.
#.##.
#.###
#####
```

Código 5: Patrón a imprimir en el problema

4. **Palíndromas:** diseñe un programa que tome como entrada una cadena de texto e imprima y responda la pregunta ¿cuántas subcadenas son palíndromas de longitud? Al menos 2 tiene la cadena leída. El programa debe mostrar todas las subcadenas palindromas solicitadas en orden lexicográfico y la cantidad de ellas.

- Descripción de entrada: una cadena de texto.
- Descripción de salida: las cadenas de texto según lo descrito en el enunciado, de **a** una por línea, y el número indicando la cantidad de cadenas mostradas en una línea separada.
- Ejemplo de entrada:
  anitalavalatinaanaeslamejor
- Ejemplo de salida:
  aa
  ala
  ala
  alavala
  ana
  anitalavalatina
  ava
  italavalati
  laval
  naan
  nitalavalatin
  talavalat
  12

5. Diseñar e implementar un programa que tome como entrada un número entero **n**, seguido a esto lea **n** números enteros; el programa debe mostrar los números leídos sin repeticiones y ordenados ascendentemente.

- Descripción de entrada: un número **n** entero, seguido de **n** números enteros, todos mayores o iguales a 0 y menores o iguales a 1000.
- Descripción de salida: los elementos leídos, sin repeticiones y ordenados ascendentemente.
- Ejemplo de entrada:
  14
  10 29 5 6 29 10 10 10 6 6 6 5 5 5
- Ejemplo de salida:
  5 6 10 29

6. Diseñar e implementar un programa que tome como entrada un número entero **n**; seguido a esto, lea **n** números enteros y diga cuántas veces está cada entero leído en la lista de enteros leídos. Adicionalmente, debe mostrar los números ordenados por frecuencia, es decir, debe primero mostrar el que más veces aparece en la entrada, después el segundo más común, después el tercero y así sucesivamente.

- Descripción de entrada: un número **n**, entero, seguido de **n** números enteros todos mayores o iguales que 0, y menores o iguales a 1000.
- Descripción de salida: la cantidad de veces que aparece cada número. Los números deben estar en orden descendente respecto a la cantidad de veces que aparece.
- Ejemplo de entrada:
  13
  10 29 5 6 29 10 2 2 2 10 2 10 2
- Ejemplo de salida:
  El 2 aparece 5 veces.
  El 10 aparece 4 veces.
  El 29 aparece 2 veces.
  El 6 aparece una vez.
  El 5 aparece una vez.

7. **Frecuencias**: diseñar e implementar un programa que tome como entrada un número entero **n**; seguido a esto, lea **n** números enteros y muestre una tabla de frecuencias de los números leídos usando asteriscos *****. Deben estar ordenados en la salida respecto a la frecuencia, estando primero los que más veces aparecen en la entrada.

- Descripción de entrada: un número **n** entero, seguido de **n** números enteros todos mayores o iguales a 0 y menores o iguales a 1000.
- Descripción de salida: la tabla de asteriscos según el ejemplo.
- Ejemplo de entrada:
  13
  10 29 5 6 29 10 2 2 2 10 2 10 2
- Ejemplo de salida:
  2 *****
  10 ****
  29 **
  6 *
  5 *

8. **Frecuencias**: diseñar e implementar un programa que tome como entrada un número entero **n**; seguido a esto lea **n** números enteros y después muestre una tabla de frecuencias de los números leídos usando asteriscos *****. Deben estar ordenados en la salida respecto a la frecuencia. La cantidad de asteriscos mostrados por número debe ser la mínima posible, siempre y cuando se conserve la proporción respecto de la cantidad de números leída. En la entrada del ejemplo expuesto, aparecen 4 asteriscos para el número 10, aunque el número 10 aparece 12 veces. Esto porque la cantidad de asteriscos por número es proporcional a la cantidad de veces que cada número se leyó en la entrada. En el ejemplo mostrado, la cantidad de números es 3 multiplicado por la cantidad de asteriscos:

12 veces el número 10, 6 veces el número 29, tres veces el número 6 y 3 veces el número 5 (12+6+3+ 3=24).

- Descripción de entrada: un número n entero, seguido de n números enteros todos mayores o iguales a 0 y menores o iguales a 1000.
- Descripción de salida: la tabla de asteriscos según el ejemplo.
- Ejemplo de entrada:
  24
  10 29 5 6 29 10 10 10 10 29 5 6 29 10 10 10 10 29 5 6 29 10 10 10
- Ejemplo de salida:
  10 ****
  29**
  6*
  5*

9. Se desea hacer un programa que tome como entrada los datos de estudiantes (nombres completos, números de documento) y una serie de notas, y produzca una lista Estudiante-Promedio ordenada de mejor a peor promedio de notas y muestre el resultado a manera de tabla con tres columnas: nombre, cédula y promedio. El formato del archivo de entrada es el siguiente:

La primera línea del archivo de entrada tendrá un número entero positivo n, que se constituye en la cantidad de líneas que siguen; cada línea representará la información de un estudiante y tendrá el siguiente formato: nombre completo - número de documento - serie de notas entre 0 y 5 separadas por espacios.

- Descripción de salida: la lista según lo solicitado.
- Ejemplo de entrada:
  4
  Rafael Alberto Velasquez-889993893-4 3 4 4.5 4 2 1
  Jose Figueroa-2334234-4 3 1 3 5
  Diego Castilla-232234-3 4 1 2 3 3 2 1 2.3 2.3
  Alberto Baron-234213-3 3 1
- Ejemplo de salida:
  Alberto Baron 234213 4.33333
  Rafael Alberto Velasquez 889993893 3.21429
  Jose Figueroa 2334234 3.2
  Diego Castilla 232234 2.36

10. **Número de Fibonacci**: para este programa se recomienda implementar la función recursiva y usar *memoization* para optimizarlo:

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-1) + f(n-2) & \text{en otro caso.} \end{cases}$$

- Descripción de entrada: un número entero positivo entre 0 y 90.
- Descripción de salida: un entero positivo el enésimo número de Fibonacci (The OEIS Foundation, 2018).
- Ejemplo de entrada:
  0
- Ejemplo de salida:
  1
- Ejemplo de entrada 2:
  1
- Ejemplo de salida 2:
  1
- Ejemplo de entrada 3:
  2
- Ejemplo de salida 3:
  2
- Ejemplo de entrada 4:
  40
- Ejemplo de salida 4:
  102334155

Referencias

Cormen, T. , Leiserson, C. , Rivest, R. y Stein, C. (2009). *Introduction to Algorithms*. Cambridge, MA, USA: The MIT Press.

Oracle. (1999). *Class TreeMap <K,V>.* Recuperado de https://docs.oracle.com/javase/7/docs/api/java/util/TreeMap.html

Stack Overflow Community. (2018). *Big-O summary for Java Collection Framework implementations?* Recuperado de https://stackoverflow.com/questions/559839/big-o-summary-for-java-collections-framework-implement

The OEIS Foundation, O. (s.f.). *The On-Line Encyclopedia of Integer Sequences: Fibonacci Numbers.* Recuperado de https://oeis.org/A000045

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 3: Principales contenedores de datos
Escenario 6: Contenedores estándar de datos no lineales

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.